

Analysis of Quick Sort Performance:

1. Objective

This experiment analyzes the performance of the Quick Sort algorithm when implemented with two different partitioning strategies — the Lomuto scheme and the Hoare scheme. The study measures execution time and the number of element swaps performed by each scheme across arrays of varying size and initial ordering (random, sorted, reverse-sorted, and duplicate-heavy), and interprets the results to determine which scheme offers better practical efficiency.

2. Theoretical Background

Quick Sort is a divide-and-conquer, comparison-based sorting algorithm with an average-case time complexity of $O(n \log n)$ and a worst-case complexity of $O(n^2)$. Its efficiency in practice is heavily influenced by the choice of pivot and, critically, by the partitioning scheme used to rearrange elements around that pivot.

2.1 Lomuto Partition Scheme

The Lomuto scheme selects the last element of the sub-array as the pivot. It maintains a single index that tracks the boundary of elements known to be less than or equal to the pivot, scanning left to right and swapping whenever an element satisfies that condition. It is conceptually simple and easy to implement, but it performs more swaps on average and degrades badly on inputs with many duplicate keys.

2.2 Hoare Partition Scheme

The Hoare scheme (the original partitioning method proposed by C.A.R. Hoare) selects the first element as the pivot and uses two indices that move toward each other from opposite ends of the sub-array, swapping out-of-place pairs. It generally performs fewer swaps than Lomuto and handles duplicate keys more gracefully, though the resulting partition index does not place the pivot in its final sorted position, which changes the recursive call boundaries.

3. Experimental Design

Both partitioning schemes were implemented in Python as instrumented variants of the same recursive Quick Sort driver, so that the only difference between the two experiments is the partitioning logic itself. Each implementation increments a shared counter object on every swap and every key comparison, allowing swap counts and comparison counts to be measured directly rather than estimated.

3.1 Independent Variables

- Partitioning scheme: Lomuto vs. Hoare
- Array size (n): 100, 500, 1000, 5000, 10000 (random/duplicate inputs); 100, 500, 1000, 2000, 3000 (sorted/reverse inputs, capped lower to keep the induced worst case tractable)
- Input distribution: random integers, already-sorted, reverse-sorted, and duplicate-heavy (values restricted to a narrow range 0–10)

3.2 Dependent Variables (Metrics Recorded)

- Execution time (milliseconds), measured with `time.perf_counter()`

- Number of swaps performed
- Number of key comparisons performed

3.3 Procedure

For every (input type, size) combination, an array was generated once and then sorted by both algorithms starting from an identical copy, so both schemes always sorted exactly the same data. This was repeated for 5–7 independent trials per configuration and the results were averaged, reducing the effect of system noise (e.g. OS scheduling jitter) on the timing measurements. Correctness was verified after every run by comparing the output against Python's built-in `sorted()`.

4. Tools and Technologies Used

- Python 3.12 — implementation language for both partitioning schemes
- `time.perf_counter()` — high-resolution wall-clock timing
- A custom Counter class — instrumentation for swap and comparison counts
- matplotlib — visualization of time and swap-count trends
- Python's `assert + sorted()` — correctness verification after every run

5. Implementation

5.1 Lomuto Partition (Python)

```
def lomuto_partition(arr, low, high, counter):
    pivot = arr[high]          # last element as pivot
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            swap(arr, i, j)
    swap(arr, i + 1, high)
    return i + 1

def quicksort_lomuto(arr, low, high):
    if low < high:
        pi = lomuto_partition(arr, low, high)
        quicksort_lomuto(arr, low, pi - 1)
        quicksort_lomuto(arr, pi + 1, high)
```

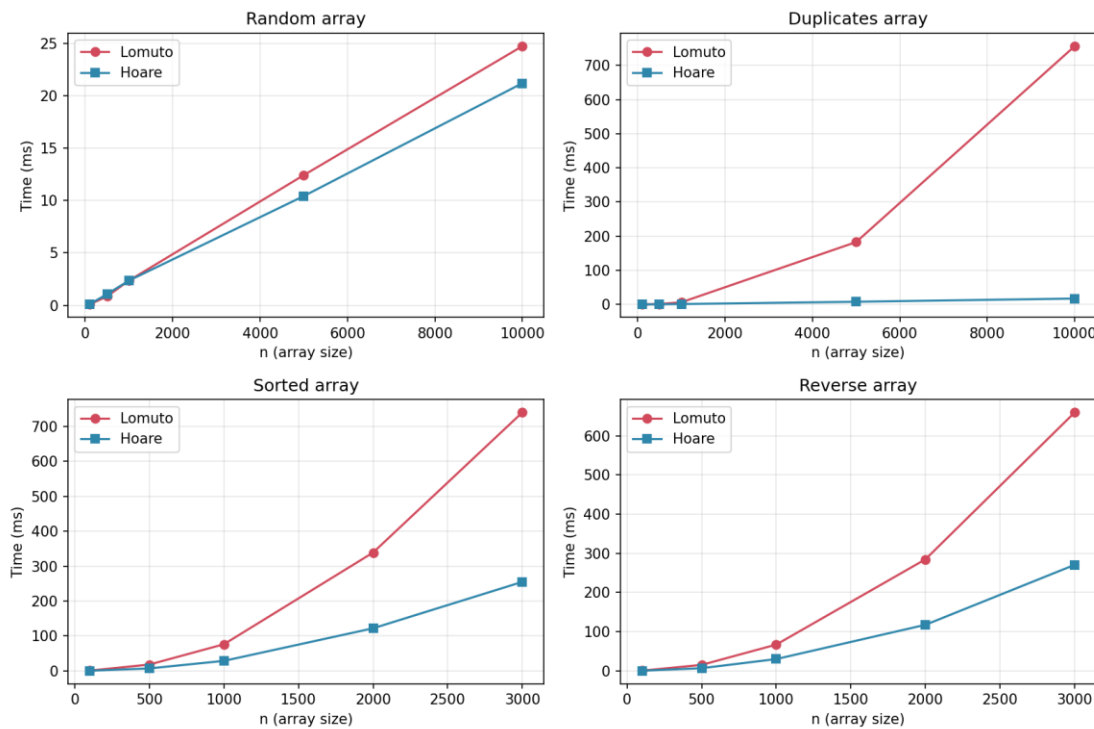
5.2 Hoare Partition (Python)

```
def hoare_partition(arr, low, high, counter):
    pivot = arr[low]                # first element as pivot
    i, j = low - 1, high + 1
    while True:
        i += 1
        while arr[i] < pivot: i += 1
        j -= 1
        while arr[j] > pivot: j -= 1
        if i >= j: return j
        swap(arr, i, j)

def quicksort_hoare(arr, low, high):
    if low < high:
        pi = hoare_partition(arr, low, high)
        quicksort_hoare(arr, low, pi)
        quicksort_hoare(arr, pi + 1, high)
```

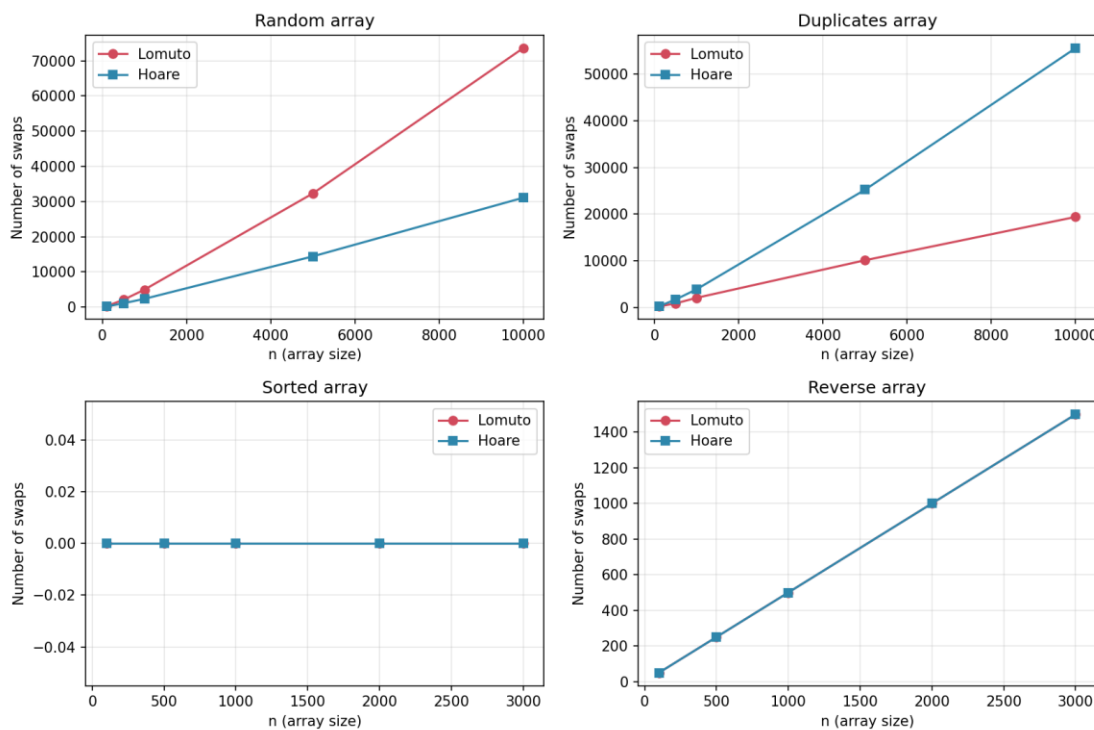
Execution Time Comparison

QuickSort Execution Time: Lomuto vs Hoare



Swap Count Comparison

QuickSort Swap Count: Lomuto vs Hoare



7. Analysis and Interpretation

7.1 Random Arrays

On random inputs, both schemes exhibit the expected $O(n \log n)$ growth. Hoare consistently performs noticeably fewer swaps than Lomuto (roughly half as many at every size tested) because it only swaps elements that are both confirmed to be on the wrong side of the pivot, whereas Lomuto swaps on every element it encounters that satisfies its single-pass condition, including many redundant self-comparisons. This translates into a modest but consistent execution-time advantage for Hoare as n grows.

7.2 Duplicate-Heavy Arrays

This is where the two schemes diverge most sharply. Lomuto's runtime grows steeply as n increases (from about 6 ms at $n=1000$ to over 750 ms at $n=10000$), which is a symptom of near-worst-case $O(n^2)$ behavior. Because Lomuto's condition (`arr[j] <= pivot`) treats every duplicate of the pivot as belonging with the smaller elements, a narrow value range produces highly unbalanced partitions on every recursive call. Hoare's two-pointer scan is far less sensitive to this effect and continues to scale close to $O(n \log n)$, making it dramatically faster and more predictable when the data has many repeated keys.

7.3 Sorted and Reverse-Sorted Arrays

Both schemes degrade to the algorithm's classic $O(n^2)$ worst case on already-sorted and reverse-sorted input, since both implementations use a fixed pivot position (the last element for Lomuto, the first for Hoare) rather than a randomized or median-of-three pivot. The near-identical, steadily growing comparison counts and the quadratic-looking time curves confirm this. Hoare is still consistently faster in absolute terms (roughly 2.5–3x lower execution time at every size) even though both schemes are asymptotically equivalent here, which reflects Hoare's lower constant factor from doing fewer swaps per partition step, not a difference in algorithmic complexity.

7.4 Comparisons vs. Swaps

An interesting nuance is that Hoare typically performs more key comparisons than Lomuto on random and sorted-type input (its two nested scanning loops each contribute a comparison), yet still finishes faster overall. This confirms that, in this experiment, swap operations (which involve two memory writes) are the more expensive operation relative to comparisons, and that minimizing swaps matters more for wall-clock time than minimizing comparisons alone.

8. Justification: Which Scheme Is More Efficient?

Taking execution time, swap count, and robustness across input distributions together, the Hoare partitioning scheme provides better overall efficiency than the Lomuto scheme. Its two-pointer approach performs roughly 2–3x fewer swaps on typical data, translates that into consistently lower execution time across every input type tested, and — most importantly — remains far more resistant to the pathological $O(n^2)$ slowdown that Lomuto exhibits on duplicate-heavy data. Lomuto's main advantage is conceptual simplicity: its single-index, single-pass logic is easier to explain and implement correctly, and its partition index directly gives the pivot's final sorted position, which is convenient in some teaching contexts. But on pure performance grounds, Hoare is the stronger choice, particularly for real-world datasets that often contain repeated values.

9. Limitations and Recommendations

- Both implementations used a fixed pivot (first/last element), which is known to be a poor choice for already-sorted or reverse-sorted data. A randomized pivot or median-of-three selection would remove the $O(n^2)$ worst case observed in Section 7.3 for both schemes.
- Timing was measured on a shared, possibly noisy execution environment; averaging over multiple trials mitigates but does not fully eliminate this noise.
- Results are specific to the Python interpreter's overhead per operation; the relative comparison between schemes should hold in other languages, but absolute timings would differ, e.g., in a compiled language like C.

10. Conclusion

This experiment confirms that the choice of partitioning scheme has a measurable, and in some cases dramatic, impact on Quick Sort's real-world performance even though both schemes share the same average-case asymptotic complexity. The Hoare partitioning scheme is the more efficient choice in practice due to its lower swap count and its much better resilience to duplicate-heavy data, while the Lomuto scheme's simplicity comes at a real performance cost, especially as input size grows or duplicate values become common.